Part 1: Single-Axis Drone Attitude Control

Modelling, PID Control, Simulink and Frequency-Domain Analysis

A practical introduction to modelling one rotational axis of a multirotor, designing a feedback controller, and analysing the same system in both the time and frequency domains.

Note: AI used in the creation of this activity, but it mirrors analysis that you will come across both in DARES and throughout control subjects in your degree. I've added my own comments in yellow - Rohan

DARES Drone Club – Flight Control

September 2026

Contents

1 Learning Objectives **2**

2 Plant Model **2**

3 Activity A – Time-Domain MATLAB Model **3**
3.1 Tasks . . . 3
3.2 MATLAB skeleton . . . 3

4 Activity B – Transfer-Function Representation **4**
4.1 Tasks . . . 4
4.2 Reference / skeleton code . . . 4

5 Activity C – Simulink Model **5**
5.1 Recommended first model: physics using integrators . . . 5
5.2 Blocks to use . . . 6
5.3 Tasks . . . 6

6 Activity D – Root Locus and Pole Analysis **6**
6.1 Tasks . . . 7

7 Activity E – Frequency-Domain Analysis **7**
7.1 Concepts to connect . . . 7
7.2 Tasks . . . 8
7.3 MATLAB skeleton . . . 8

8 Activity F – Bring the Views Together **9**

9 Useful MATLAB Commands – Quick Reference **9**

10 Extension Challenges **9**

1 Learning Objectives

By the end of this activity you should be able to:

- derive a simple rotational model from Newton's second law for rotation;
- simulate the model numerically in MATLAB;
- construct the same plant using a transfer function and in Simulink;
- explain the physical effects of proportional, derivative and integral feedback;
- use poles, root locus, Bode plots and stability margins to analyse a controller;
- connect time-domain behaviour such as rise time and oscillation with frequency-domain concepts such as bandwidth and phase margin;
- recognise the limitations of the simplified model before applying ideas to a real aircraft.

2 Plant Model

Derived from Newton's second law



Figure 1: Example System

Consider one rotational axis, pitch θ (as seen in Figure 1). For the introductory model,

$$J\ddot{\theta} = \tau + d, \tag{1}$$

where J is the pitch moment of inertia, τ is commanded control torque, and d is an external disturbance torque. Hence

$$\ddot{\theta} = \frac{\tau + d}{J}. \tag{2}$$

Use the nominal value

$$J = 0.02 \text{ kg m}^2.$$

Assuming zero initial conditions, the transfer function (taking Laplace transform) from torque to attitude is

$$P(s) = \frac{\Theta(s)}{\mathcal{T}(s)} = \frac{1}{Js^2}. \tag{3}$$

An optional slightly more realistic model includes rotational damping b (including friction/drag):

$$J\ddot{\theta} + b\dot{\theta} = \tau, \qquad P(s) = \frac{1}{Js^2 + bs}. \tag{4}$$

Key idea

The undamped plant contains two poles at the origin. A constant torque produces constant angular acceleration, so angle grows quadratically rather than settling to a constant value. Think about taking the double integral of $\ddot{\theta}$ with respect to time

3 Activity A – Time-Domain MATLAB Model

Objective: Build the dynamics directly from the differential equation before relying on Control System Toolbox functions.

3.1 Tasks

1. Apply a constant torque with no feedback. Plot $\theta(t)$ and $\dot{\theta}(t)$ and explain their shapes.
2. Add proportional feedback think about what happens to the sign and size of input torque when the angle is above vs below the target

$$\tau = K_P(\theta_{\text{ref}} - \theta).$$

   Command $0° \rightarrow 10°$ and test several values of K_P.
3. Add derivative feedback using angular rate, think about what happens to the sign and size of input torque when the angle is changing quickly/slowly, and positive/negative

$$\tau = K_P e - K_D \dot{\theta}.$$

   Compare the response with P-only control.
4. Add integral action, think about what happens to the sign and size of input torque when the angle has been above the target for a long/short time, vs below the target for a long/short time

$$\tau = K_P e + K_I \int e\, dt - K_D \dot{\theta}.$$

5. Apply a temporary disturbance torque and compare P, PD and PID disturbance rejection.
6. Add torque saturation, for example $|\tau| \leq 0.20$ N m, and observe the effect of an aggressive reference command. think about why we need to include saturation constraints on our real controller

Hints

Use radians internally and convert only for plotting. Start with $T_s = 0.005$ s (sample time, we can't send continuous signals as our real microcontroller runs in digital loops. Tune one term at a time. Do not search only for a "good" response: deliberately create slow, oscillatory and saturated cases and explain them.

3.2 MATLAB skeleton

```
%% Part 1A - direct numerical simulation
clear; clc; close all;

Ts = 0.005;
t = 0:Ts:5; %setting up 5 second time scale
N = length(t); %number of time steps

J = 0.02; % kg m^2
tauMax = 0.20; % N m
ref = deg2rad(10)*ones(1,N); %setting up reference for each time step

Kp = ...;
Ki = ...;
Kd = ...;

```

```
%setting up empty vectors for our variables at each time step
theta = zeros(1,N);
omega = zeros(1,N);
tau = zeros(1,N);
Ierr = 0; %integral total error over time

for k = 1:N-1
    e = ref(k) - theta(k);
    Ierr = Ierr + e*Ts;

    tauCmd = Kp*e + Ki*Ierr - Kd*omega(k); %commanded torque
    tau(k) = max(min(tauCmd,tauMax),-tauMax); %introducing saturation

    disturbance = 0;
    % if t(k) >= 2 && t(k) < 2.2
    % disturbance = 0.05;
    % end

    alpha = (tau(k) + disturbance)/J; %comes from model
    omega(k+1) = omega(k) + alpha*Ts; %approximate integration - think about why this works
    theta(k+1) = theta(k) + omega(k+1)*Ts; %approximate integration
end

figure;
plot(t,rad2deg(theta),'LineWidth',1.3); hold on;
plot(t,rad2deg(ref),'--','LineWidth',1.2);
grid on;
xlabel('Time [s]'); ylabel('Pitch angle [deg]');
legend('theta','theta ref');

figure;
plot(t,tau,'LineWidth',1.3); grid on;
xlabel('Time [s]'); ylabel('Torque [N m]');
```

4 Activity B – Transfer-Function Representation

Objective: Represent the same physical model as $P(s)$ and use MATLAB's linear-system analysis tools .you will be prompted to install MATLAB's control systems toolbox. If you don't know what a command is doing, read up on the MATLAB documentation for it, then check your understanding with me/AI

4.1 Tasks

1. Construct $P(s) = 1/(Js^2)$ using both `tf('s')` and coefficient-vector syntax.
2. Use `pole`, `zero` and `pzmap`. Relate the two poles at the origin to the open-loop time response.
3. Form a unity-feedback proportional controller using `feedback` and compare the toolbox step response with your direct simulation.
4. Construct PD and PID controllers using both algebraic transfer functions and MATLAB's `pid` object.
5. Use `stepinfo` to record rise time, settling time and overshoot for selected stable controllers.

4.2 Reference / skeleton code

```m
%% Part 1B - transfer-function model
clear; clc; close all;
J = 0.02;
s = tf('s');

P = 1/(J*s^2); %our Plant (system model)
P_alt = tf(1,[J 0 0]); %second way of expressing the same thing

pole(P)
zero(P)
figure; pzmap(P); grid on;

% Proportional controller
Kp = ...;
C_P = Kp; %purely proportional gain
L_P = C_P*P; % open-loop transfer function
T_P = feedback(L_P,1); % closed-loop ref -> output
figure; step(T_P); grid on;

% PD controller
Kd = ...;
C_PD = Kp + Kd*s;
T_PD = feedback(C_PD*P,1);
figure; step(T_PD); grid on;
stepinfo(T_PD)

% PID object alternative using MATLAB's built in PID
Ki = ...;
C_PID = pid(Kp,Ki,Kd);
T_PID = feedback(C_PID*P,1);
```

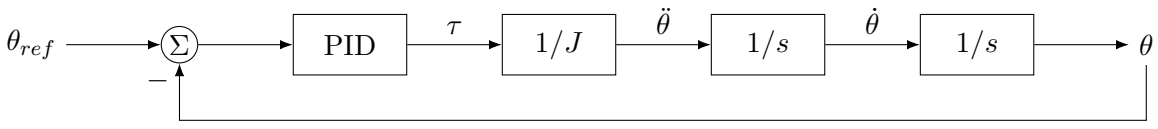
5 Activity C – Simulink Model

Objective: Build the feedback loop graphically and verify that it represents the same equations as the MATLAB model. launch from MATLAB, double click to add a block

5.1 Recommended first model: physics using integrators

Build the plant from the equation $\ddot{\theta} = \tau/J$ rather than starting with a Transfer Fcn block. A useful signal flow is

$$\tau \longrightarrow \frac{1}{J} \longrightarrow \ddot{\theta} \longrightarrow \frac{1}{s} \longrightarrow \dot{\theta} \longrightarrow \frac{1}{s} \longrightarrow \theta.$$



5.2 Blocks to use

Block	Purpose
Step	attitude reference θ_{ref}
Sum	form $e = \theta_{ref} - \theta$
PID Controller	implement P/PD/PID feedback
Saturation	impose the torque limit
Gain	implement $1/J$
Integrator (two)	integrate acceleration to rate and rate to angle
Scope	inspect reference, angle, rate and torque
To Workspace	export signals for MATLAB analysis
Transfer Fcn	alternative implementation of $1/(Js^2)$

5.3 Tasks

1. Build the integrator-based plant and first run it open loop.
2. Close the loop with a P controller, then PD, then PID.
3. Add a Saturation block immediately after the controller.
4. Add a disturbance torque with a Step/Pulse signal summed into the plant input after the controller.
5. Build a second version using a Transfer Fcn block with numerator [1] and denominator [J 0 0]. Confirm that it matches the integrator model when saturation/nonlinearities are absent.
6. Export θ, θ_{ref} and τ to MATLAB. Overlay the Simulink result with your MATLAB simulation.

Hints

Set model variables such as `J`, `Kp`, `Ki` and `Kd` in a MATLAB setup script rather than hard-coding numbers into every block. Use the same solver/sample assumptions when comparing MATLAB and Simulink. For the first continuous Simulink model, a variable-step solver is fine; later, try a fixed-step solver to see the connection with embedded implementation.

6 Activity D – Root Locus and Pole Analysis

Objective: Use pole locations to explain why the time response changes as controller structure and gain change. Techniques to analyse/predict behaviour before simulation. Brian Douglas has videos on these as well.

For P control,

$$1 + K_P P(s) = 0 \quad \Rightarrow \quad Js^2 + K_P = 0,$$

so

$$s = \pm j\sqrt{\frac{K_P}{J}}.$$

Thus ideal P-only feedback moves the poles along the imaginary axis: increasing K_P changes oscillation frequency but does not create damping.

For a PD controller $C(s) = K(s + z)$, the added controller zero changes the root-locus geometry and allows closed-loop poles to move into the left-half plane.

6.1 Tasks

1. Plot `rlocus(P)` and explain the result.
2. Choose a PD zero, e.g. $z = 5$, and plot `rlocus((s+z)*P)`.
3. Use `sgrid` to overlay damping-ratio/natural-frequency lines.
4. Pick two or three points on the locus using `rlocfind` or manually selected gains. Compare their pole locations and step responses.
5. Predict which response should be faster/more oscillatory before running `step`.

```
%% Part 1D - root locus
s = tf('s');
J = 0.02;
P = 1/(J*s^2);

figure;
rlocus(P); grid on;
title('P-control root locus');

z = 5;
Cshape = s + z; % gain K is varied by rlocus
figure;
rlocus(Cshape*P); grid on;
sgrid;
title('PD root locus');

% Optional interactive gain selection
% [K,poles] = rlocfind(Cshape*P);
% T = feedback(K*Cshape*P,1);
% figure; step(T); grid on;
```

7 Activity E – Frequency-Domain Analysis

Objective: Analyse the open-loop system $L(s) = C(s)P(s)$ using Bode plots, crossover frequency, gain margin and phase margin.

7.1 Concepts to connect

- **Magnitude:** how strongly the loop responds to signals at each frequency.
- **Phase:** how much phase lag the loop introduces.
- **Gain crossover frequency ω_c:** frequency where $|L(j\omega)| = 1$ (0 dB).
- **Phase margin:** additional phase lag required at gain crossover to reach $-180°$.
- **Gain margin:** factor by which loop gain can increase at the phase-crossover frequency before the classical stability boundary is reached.
- **Bandwidth:** roughly indicates how quickly the closed loop can respond; higher bandwidth generally means faster response but also greater exposure to high-frequency noise/unmodelled dynamics.

Key idea

Always analyse margins using the *open-loop* transfer function $L(s) = C(s)P(s)$, not the closed-loop transfer function $T(s)$. Then compare those frequency-domain observations with the closed-loop step response.

7.2 Tasks

1. Plot the Bode diagram of the plant $P(s)$. Explain the low-frequency/high-frequency magnitude trend and phase.
2. Plot `margin(L)` for a stable PD controller. Record gain margin, phase margin and crossover frequencies using `margin` or `allmargin`.
3. Change K_P while keeping the controller structure fixed. Observe how crossover frequency and phase margin change.
4. Change derivative action / the PD zero and observe the phase contribution around crossover.
5. Plot the closed-loop complementary sensitivity

$$T(s) = \frac{L(s)}{1 + L(s)}$$

and sensitivity

$$S(s) = \frac{1}{1 + L(s)}.$$

Verify numerically that $S + T = 1$.

6. Use `bandwidth(T)` and compare bandwidth with the time-domain speed of the step response.
7. Optional: add a first-order actuator or sensor lag and observe the loss of phase margin. Explain why real systems cannot be designed from $1/(Js^2)$ alone.

7.3 MATLAB skeleton

```
%% Part 1E - frequency-domain analysis
clear; clc; close all;
s = tf('s');
J = 0.02;
P = 1/(J*s^2);

% Choose a stable controller
Kp = ...;
Kd = ...;
C = Kp + Kd*s;
L = C*P;
T = feedback(L,1);
S = feedback(1,L); % 1/(1+L)

figure;
bode(P); grid on;
title('Plant Bode plot');

figure;
margin(L); grid on;
title('Open-loop stability margins');

[Gm,Pm,Wcg,Wcp] = margin(L)
AM = allmargin(L)

BW = bandwidth(T)

figure;
bode(S,T); grid on;
legend('S = 1/(1+L)','T = L/(1+L)');

```

```
% Optional actuator lag
wm = 40; % example actuator pole [rad/s]
Gact = wm/(s + wm);
L_lag = C*P*Gact;
figure;
margin(L_lag); grid on;
title('Margins after adding actuator lag');
```

8 Activity F – Bring the Views Together

For at least three controller choices, complete a small comparison table. Do not optimise for a particular target; the aim is to explain relationships.

Controller	Dominant poles	Overshoot	Settling time	Phase margin	Bandwidth
P					
PD – case 1					
PD/PID – case 2					

Answer the following in a few sentences each:

1. Why does P-only control fail to provide damping for the ideal $1/(Js^2)$ plant?
2. What does derivative action change in both the time-domain and frequency-domain views?
3. What is the trade-off when increasing closed-loop bandwidth?
4. Why can a controller that looks excellent on this simplified model still perform poorly or become unstable on a real drone?
5. Which effects would you add next to improve the model? Consider motor/propeller dynamics, delays, sensor filtering, aerodynamic damping, axis coupling, saturation and discrete sampling.

9 Useful MATLAB Commands – Quick Reference

Command	Use
`tf, zpk`	Create transfer-function / zero-pole-gain models.
`pole, zero, pzmap`	Inspect poles and zeros.
`feedback`	Form a closed-loop interconnection.
`step, stepinfo`	Time-domain step response and metrics.
`rlocus, rlocfind, sgrid`	Root-locus analysis and gain selection.
`bode, bodeplot`	Frequency response.
`margin, allmargin`	Classical gain/phase stability margins.
`bandwidth`	Closed-loop bandwidth.
`pid`	Construct P/PI/PD/PID controllers.
`minreal`	Remove cancelling pole-zero pairs when appropriate.
`lsim`	Simulate arbitrary time-domain inputs.
`linearize`	Linearise a suitable Simulink model (advanced/optional).

10 Extension Challenges

If the core activity is comfortable:

1. Add a motor/propeller first-order lag $G_m(s) = \omega_m/(s + \omega_m)$ and compare root locus and margins.

2. Add a pure computational delay approximation using `pade`. Observe its effect on phase margin.
3. Compare continuous and discrete controllers with `c2d` for several sample times.
4. Linearise a more detailed Simulink model around hover and compare its linear response with the simple analytical model.
5. Replace the angle derivative in the controller with a noisy gyro measurement and investigate filtering versus phase lag.